
Durin's Code

Language Reference Manual

CS4031 — Compiler Construction
Spring 2026

Student Name	Student ID
Huzaifa Abdul Rehman	23K-0782
Abdul Moiz Hussain	23K-0553
Muhammad Abdullah Khan	23K-0607

Contents

1	Introduction	2
2	Lexical Structure	2
2.1	Source Encoding	2
2.2	Comments	2
2.3	Identifiers	2
2.4	Keywords	2
2.5	Literals	3
2.6	Operators and Punctuation	3
2.7	Context-Sensitive Dot Tokens	3
3	Grammar (EBNF)	3
4	Declarations	5
4.1	Room Declaration	5
4.2	Action Declaration	5
5	Statements	6
5.1	print	6
5.2	remove	6
5.3	Assignment	6
5.4	if / else	6
6	Conditions	7
7	Type System	7
8	Scope Rules	7
9	Error Catalogue	7
9.1	Lexer Errors	8
9.2	Semantic Errors	8
10	Complete Example Program	8

1 Introduction

Durin's Code (file extension `.dc`) is a domain-specific language designed for authoring interactive text-adventure games in the spirit of the 8-bit era. A `.dc` program declares a *world* — a directed graph of rooms populated with items and NPCs — and a set of *actions*, which are named event handlers triggered by player input at runtime.

The compiler performs full static validation of the world graph, generates optimised Three-Address Code (TAC) as an intermediate representation, serialises the result to JSON bytecode, and executes it inside a terminal-based Virtual Machine.

Design philosophy. Durin's Code removes the need for imperative low-level coding when building text adventures. Every game element (rooms, items, NPCs, transitions) is expressed declaratively. All cross-references are validated at compile time, preventing runtime crashes from missing rooms or invalid items.

2 Lexical Structure

2.1 Source Encoding

Source files are plain text encoded in UTF-8. Both Unix (`\n`) and Windows (`\r\n`) line endings are accepted; the lexer normalises them to a single newline internally.

2.2 Comments

Only single-line comments are supported. A comment begins with `//` and extends to the end of the current line.

```

1 // This is a comment. It is ignored by the compiler.
2 room "start" {    // inline comments are also allowed
3     description "A quiet room."
4 }
```

Listing 1: Comment syntax

2.3 Identifiers

$$\textit{identifier} ::= [_\text{a-zA-Z}] [_\text{a-zA-Z0-9}]^*$$

Identifiers are case-sensitive. A leading underscore is permitted. Examples: `the_ring`, `_private`, `room1`, `hasItem`.

2.4 Keywords

The following identifiers are reserved and may not be used as user-defined names:

room	action	item	npc
exit	if	else	print
remove	player	true	false
description	current_room		

2.5 Literals

String literals. A string is enclosed in double-quotes on a single line.

$$STRING ::= " [\backslash^n]^* "$$

Integer literals. A sequence of decimal digits. Floating-point numbers are **not** supported and produce a compile-time error.

$$NUMBER ::= [0-9]^+$$

Boolean literals. `true` and `false`.

2.6 Operators and Punctuation

Symbol	Name	Usage
{ }	Braces	Block delimiters
()	Parentheses	Function-call arguments
,	Comma	Property separator
:	Colon	Property value separator
;	Semicolon	Optional statement terminator
.	Dot	Attribute access (<code>player.inventory</code>)
=	Assign	Simple assignment
==	Equal	Equality test
+=	Plus-assign	Add to attribute / add item to inventory
-=	Minus-assign	Subtract from attribute / remove from inventory
+ -	Arithmetic	Integer arithmetic
> >= < <=	Relational	Numeric comparisons
&&	Logical AND	Both conditions must hold
	Logical OR	Either condition must hold

2.7 Context-Sensitive Dot Tokens

After a `.` the lexer inspects the immediately preceding token to disambiguate attribute access:

- `player.attr` $\Rightarrow$ next identifier emitted as `TOKEN_PLAYER_ATTR`
- `room_id.attr` $\Rightarrow$ next identifier emitted as `TOKEN_ROOM_ATTR`

This allows the parser to differentiate `player.has_item(...)` from `room.description` without extra lookahead.

3 Grammar (EBNF)

The complete context-free grammar of Durin's Code in Extended Backus-Naur Form:

Listing 2: Complete EBNF Grammar

```

program          ::=  declaration* EOF

declaration      ::=  room_decl | action_decl

(* World *)
room_decl       ::=  "room" STRING "{" room_body* "}"
room_body       ::=  description_stmt
                    | item_decl
                    | npc_decl
                    | exit_decl

description_stmt ::=  "description" STRING

item_decl       ::=  "item" IDENTIFIER "{" prop_list "}"
npc_decl        ::=  "npc"  IDENTIFIER "{" prop_list "}"
prop_list       ::=  prop ("," prop)*
prop            ::=  IDENTIFIER ":" (NUMBER | STRING | BOOLEAN)

exit_decl       ::=  "exit" IDENTIFIER STRING
                (*      ^direction  ^target_room_id *)

(* Actions *)
action_decl     ::=  "action" STRING "{" statement* "}"

statement       ::=  print_stmt
                    | remove_stmt
                    | assign_stmt
                    | if_stmt

print_stmt      ::=  "print" STRING
remove_stmt     ::=  "remove" IDENTIFIER
assign_stmt     ::=  player_attr_ref ("=" | "+=" | "-=")
                    (IDENTIFIER | NUMBER | STRING | BOOLEAN)
player_attr_ref ::=  "player" "." IDENTIFIER

if_stmt         ::=  "if" condition "{" statement* "}"
                    ( "else" "{" statement* "}" )?

(* Conditions *)
condition       ::=  simple_cond ( "&&" | "||" ) simple_cond)*
simple_cond      ::=  room_check | has_item_check | comparison

room_check      ::=  "current_room" "==" STRING
has_item_check  ::=  "player" "." "has_item" "(" IDENTIFIER ")"
comparison      ::=  player_attr_ref
                    ("==" | ">" | ">=" | "<" | "<=")
                    (NUMBER | STRING | BOOLEAN)

(* Terminals *)
STRING         ::=  ''' [^"\n]* '''

```

NUMBER	::=	[0-9]+
BOOLEAN	::=	"true" "false"
IDENTIFIER	::=	[_a-zA-Z] [_a-zA-Z0-9]*

4 Declarations

4.1 Room Declaration

A room declaration defines one node in the world graph.

```
1 room "bag_end" {  
2     description "The cozy hole of a Hobbit. A golden ring  
   glints on the table."  
3     item the_ring { power: 100, type: "artifact" }  
4     npc gandalf { health: 500, hostile: false }  
5     exit east "buckland"  
6     exit north "shire"  
7 }
```

Listing 3: Room declaration syntax

- **description** — Sets the text printed by the `look` command. Exactly one description per room.
- **item** — Places a named item in this room with a property map. The item is globally registered and may be referenced in any action.
- **npc** — Places a named NPC in the room with a property map.
- **exit** — Declares a directed edge; direction is a free identifier (`north`, `east`, or any word).

Semantic constraints.

1. Room identifiers (the string name) must be unique across the program.
2. Exit target strings must reference a declared room.
3. No two exits in the same room may share the same direction identifier.

4.2 Action Declaration

An action is a named event handler triggered by matching player input.

```
1 action "take ring" {  
2     if current_room == "bag_end" {  
3         player.inventory += the_ring  
4         print "The Precious is yours."  
5     }  
6 }
```

Listing 4: Action declaration syntax

Action names are strings and may contain spaces. At runtime the player types the action name verbatim (case-sensitive).

5 Statements

5.1 print

Writes a string literal to standard output.

```
print "You cannot do that here."
```

5.2 remove

Removes a named item from the game world and from the player's inventory if present.

```
remove the_ring
```

Semantic constraint: the identifier must name a declared item.

5.3 Assignment

Sets or modifies a player attribute.

```
player.win      = true           // boolean assignment
player.health   = 100            // integer assignment
player.name     = "Bilbo"        // string assignment
player.inventory += the_ring     // add item to inventory
player.health   -= 10            // subtract from integer
    attribute
```

Listing 5: Assignment variants

5.4 if / else

Conditional execution. The `else` branch is optional.

```
1 if current_room == "mount_doom" && player.has_item(the_ring) {
2     remove the_ring
3     player.win = true
4     print "Middle-earth is saved!"
5 } else {
6     print "You cannot do that here."
7 }
```

Listing 6: Conditional statement

6 Conditions

Form	Meaning
<code>current_room == "id"</code>	Player is in room "id"
<code>player.has_item(name)</code>	Player's inventory contains <code>name</code>
<code>player.attr == value</code>	Attribute equality
<code>player.attr > value</code>	Greater-than comparison
<code>player.attr >= value</code>	Greater-or-equal
<code>player.attr < value</code>	Less-than comparison
<code>player.attr <= value</code>	Less-or-equal
<code>cond1 && cond2</code>	Logical AND
<code>cond1 cond2</code>	Logical OR

7 Type System

Property values in `item` and `npc` blocks must be one of three scalar types:

Type	Example	Storage
Integer	100	32-bit signed
String	"artifact"	<code>std::string</code>
Boolean	<code>true</code> / <code>false</code>	<code>bool</code>

Player attributes are dynamically typed: the first assignment determines the type. Floating-point numbers are rejected at lex time.

8 Scope Rules

- All room, item, NPC, and action names share a single flat global namespace.
- Duplicate names within the same category are a semantic error.
- Items declared inside any room are globally visible in all actions.
- `player` is a built-in identifier; `player.X` accesses runtime state.
- There is no block-level scoping — no local variables exist in actions.

9 Error Catalogue

9.1 Lexer Errors

Message	Cause
Unexpected character	A character outside the language alphabet (e.g. @)
Unterminated string.	A " opened but not closed before end-of-line
Floating-point numbers are not supported.	A digit sequence containing .
Expected '&&'	Single & not followed by &
Expected ' '	Single not followed by

9.2 Semantic Errors

Error	Cause
Duplicate room	Two rooms share the same name
Duplicate item	Two items share the same name
Duplicate action	Two actions share the same name
Exit target not declared	An exit points to a non-existent room
Duplicate exit direction	A direction is used twice in the same room
remove: unknown item	Removal of an undeclared item
has_item: unknown item	Inventory check against an undeclared item
current_room: unknown room	Room comparison against an undeclared room

10 Complete Example Program

```

1 // Middle-earth adventure
2
3 room "bag_end" {
4     description "The cozy hole of a Hobbit. A golden ring
5         glints on the table."
6     item the_ring { power: 100, type: "artifact" }
7     exit east "buckland"
8 }
9
10 room "buckland" {
11     description "A quiet riverside village. The old ferry
12         creaks at the dock."
13     exit west "bag_end"
14     exit east "mount_doom"
15 }
16
17 room "mount_doom" {
18     description "The air is thick with ash. The fiery Cracks of
19         Doom loom ahead."
20     npc sauron { health: 999, hostile: true }
21     exit west "buckland"

```

```
19 }
20
21 action "take ring" {
22     if current_room == "bag_end" {
23         player.inventory += the_ring
24         print "The Precious is yours."
25     } else {
26         print "There is nothing of value here."
27     }
28 }
29
30 action "destroy ring" {
31     if current_room == "mount_doom" && player.has_item(the_ring
32     ) {
33         remove the_ring
34         print "The ring is consumed by fire! Middle-earth is
35         saved."
36         player.win = true
37     } else {
38         print "You cannot do that here."
39     }
40 }
```

Listing 7: Middle-Earth Adventure — full demo program